QA_Agent Production Deployment Plan

Deploy QA_Agent to AWS EC2 for Organization-Wide Access

Prepared	HPI Dashboard Project
Date	18 May 2026
Platform	AWS EC2 (private VPC) · Flask · Anthropic Claude API
Security	Corporate VPN / network boundary · AWS SSM for API key

Context

The HTML dashboards embed a QA chat widget (**QA_Agent/widget.py**) that currently calls **http://localhost:5050/ask** — a Flask server that must be running on the *same machine* as the browser. When users in the organization receive the HTML files, the QA_Agent silently fails because no server is running on their machines.

The fix is to deploy the existing Flask server once, centrally, on AWS EC2 inside the corporate VPC (accessible over VPN), then update one URL in the widget so all rebuilt dashboards point to the shared server. The Anthropic API key is held only on the EC2 instance — never embedded in the HTML files.

Architecture After This Change

```
User's machine (file://*.html)
        ■
        ■  POST http://<ec2-private-ip>:5050/ask
        ▼  (reachable only via corporate VPN / corporate network)
AWS EC2 t3.small — private subnet
  ■■■ python QA_Agent/server.py  (systemd service, port 5050)
        ■  Anthropic Python SDK
        ▼
Anthropic Claude API  (claude-haiku-4-5-20251001)
```

HTML files remain self-contained static files — no web server required to view them. The only network call they make is POST /ask to the EC2 backend.

Code Changes Required

1. QA_Agent/widget.py (line ~6)

Replace the hardcoded localhost URL with an environment-variable lookup:

```
# Before
QA_URL = "http://localhost:5050/ask"

# After
import os
QA_URL = os.environ.get("QA_AGENT_URL", "http://localhost:5050") + "/ask"
```

Setting **QA_AGENT_URL** in the shell before running the build script bakes the correct URL into every generated dashboard.

2. QA_Agent/server.py — CORS (already configured)

flask-cors is already enabled. Verify **CORS(app)** has no origin restriction. file:// dashboards carry no Origin header; flask-cors handles this correctly with its default settings — no change needed.

3. New deployment files QA_Agent/deploy/

File	Purpose
ec2_setup.sh	One-shot setup: installs Python 3.11, pip deps, copies QA_Agent/
hpi-qa-agent.service	systemd unit — auto-starts server on reboot, references API key path
ssm_param.md	Instructions: store ANTHROPIC_API_KEY in AWS SSM Parameter Store

Files to Modify Summary

File	Change
QA_Agent/widget.py line ~6	Replace hardcoded URL with env-var pattern
QA_Agent/deploy/ec2_setup.sh	NEW — EC2 bootstrap shell script
QA_Agent/deploy/hpi-qa-agent.service	NEW — systemd unit file

build_dashboards_all.py and build_dashboard.py require NO changes — they call qa_widget() which reads the updated widget.py.

Deployment Steps

Phase 1 — AWS Infrastructure (one-time)

Step 1: Launch EC2 instance

- AMI: Amazon Linux 2023
- Type: t3.small (≈ $15 / month)
- VPC: private subnet reachable from corporate VPN
- Security Group: allow TCP 5050 inbound from corporate IP range ONLY
- No public IP required

Step 2: Store API key in AWS SSM Parameter Store

```
aws ssm put-parameter \
  --name /hpi/qa-agent/anthropic-api-key \
  --value "sk-ant-..." \
  --type SecureString
```

The key never touches disk in plain text and never appears in the HTML files.

Step 3: Run setup script on EC2 (SSH or AWS Session Manager)

```
bash ec2_setup.sh
```

Step 4: Start and enable the service

```
sudo systemctl enable hpi-qa-agent
sudo systemctl start  hpi-qa-agent
curl http://localhost:5050/health   # expected: {"status":"ok"}
```

Step 5: Note the EC2 private IP address (e.g. **10.0.1.45**).

Phase 2 — Rebuild Dashboards with New URL

```
$env:QA_AGENT_URL = "http://10.0.1.45:5050"
python build_dashboards_all.py
```

The four rebuilt HTML files in **20260518/** will call http://10.0.1.45:5050/ask instead of localhost.

Phase 3 — Distribute the Rebuilt HTML Files

No change to distribution method. Share the four HTML files via email or SharePoint as before. Users connected to VPN (or the corporate network) open the file in any browser — the QA chat widget works automatically.

Security Controls

Control	Implementation
Network boundary	EC2 Security Group: port 5050 from corporate IP range only
API key protection	Stored in AWS SSM Parameter Store; injected as env var; never in HTML
Authentication	VPN / corporate network boundary is the auth layer — no tokens needed
CORS	flask-cors default (permissive for file:// — no Origin header sent)
Rate limiting (future)	Add flask-limiter: 100 req/hour per IP if abuse is a concern

---

Verification

Server-side health check (EC2 terminal)

```
curl -s http://localhost:5050/health
# expected: {"status":"ok"}

curl -s -X POST http://localhost:5050/ask \
  -H 'Content-Type: application/json' \
  -d '{"question":"What is the average price in Greater Manchester?"}'
# expected: {"answer":"...","session_id":"..."}
```

End-to-end test from a VPN-connected machine

- Open rebuilt CEO_HPIAdmins.html in a browser
- Click the 'Ask the Data' floating button
- Type a question — should receive an answer within a few seconds

Negative test (outside VPN)

Opening the same HTML file from outside the VPN should display: ■ **Could not reach the QA Agent server.** — confirming the network boundary is enforced.

What Users Need To Do

Nothing. Users open the HTML file as normal. If they are on VPN or the corporate network the QA_Agent works automatically. No Python installation, no API key, no local server setup required on their machines.

Who	Action required
IT / DevOps	Deploy EC2 instance once (Phases 1-2 above)
Dashboard publisher	Set QA_AGENT_URL env var and re-run build_dashboards_all.py
End users	Open HTML file as usual — QA_Agent works if on VPN